

Retrospective

Olukorede Oduniyi

December 2025

1 Retrospective Report

1.1 Timeline Comparison

This section compares the original project timeline outlined in the project plan with the actual progress achieved during Term One.

- **Week 1 Planned:** Initial research and project planning. **Actual:** Completed as planned, including an initial meeting with the supervisor to refine the project scope. However, deciding on the final direction of the project took longer than expected.
- **Week 2 Planned:** Frontend, backend, and database design. **Actual:** The majority of this week was spent completing and refining the project plan based on supervisor feedback. Design work was delayed due to over-committing to planning and underestimating the time required to finalise the proposal.
- **Weeks 3–4 Planned:** Database creation, model design, and a simple homepage. **Actual:** The repository was set up and configured. Initial frontend designs and database drafts were produced, and both the Spring Boot backend and React frontend were scaffolded. Significant time was spent configuring tooling, CI, and project structure, which took longer than anticipated.
- **Weeks 5–6 Planned:** Asset additions and REST API integration between frontend and backend. **Actual:** Instead of feature development, this period focused heavily on backend setup and quality assurance. Test Driven Development was adopted, with entities, DTOs, mappers, repositories, and failing tests implemented. Static analysis, test coverage enforcement, and CI pipelines were also configured. This resulted in falling behind the original schedule, partly due to competing coursework for different modules.
- **Week 7 Planned:** Research into charting and live data integration. **Actual:** Backend development continued with controller and service implementation tests. Authentication and application structure were prioritised

following supervisor feedback. As a result, minimal progress was made on charting research at this stage.

- **Weeks 8–9 Planned:** Dashboard development, chart integration, testing, and user acceptance testing. **Actual:** Significant progress was made in both frontend and backend. Security configuration, Firebase authentication, protected routes, and token filtering were implemented. Full public and private UI pages were developed, including dashboard, portfolio, assets, goals, and news. The interim report template was also created and reviewed with supervisor feedback.
- **Weeks 10–11 Planned:** Interim report and presentation preparation. **Actual:** Report writing and presentation preparation were prioritised. End-to-end functionality for stocks, cryptocurrencies, and goals was completed, including backend integration and interactive frontend views. Some additional development was required to meet core Term One objectives, but focus shifted towards documentation and presentation delivery.

Conclusion Overall, the planned timeline was ambitious and, in several areas, unrealistic. In the early weeks, time was overcommitted due to underestimating the effort required to finalise the project plan and fully configure the development environment. This resulted in delays to frontend design and end-to-end functionality. Additional complexity arose from adopting Test Driven Development and implementing secure authentication, which significantly increased backend workload beyond what was originally planned.

Despite these setbacks, steady progress was made through sustained effort, particularly in the latter half of Term One. By prioritising core functionality, authentication, and system integration, the project recovered much of the initial delay. While not all planned features were completed in the originally intended order, the essential components of the system were successfully implemented.

This experience highlighted the importance of realistic planning, especially when balancing multiple modules alongside a large technical project. In future stages, greater emphasis will be placed on incremental milestones, buffer time for complex tasks, and avoiding overcommitment. These lessons will directly inform Term Two planning and reduce the likelihood of similar delays.

1.2 Term Two Plans

The second term will focus on completing the system, improving overall quality, and conducting evaluation. The following milestones are planned:

- **Weeks 1–2:** Complete all remaining backend CRUD functionality and implement edit and delete operations for assets and financial goals.

- **Weeks 3–4:** Integrate real-time asset price data and enhance data visualisation across portfolio views, asset pages, and charts.
- **Week 5:** Research the use of artificial intelligence techniques to analyse user assets and identify relevant financial news, alongside beginning work on the final project report and documentation.
- **Weeks 6–7:** Implement a personalised news feature that presents content related to a user’s tracked assets.
- **Weeks 8–9:** Refine the user interface, conduct additional testing, improve accessibility, and enhance responsiveness across devices.
- **Week 9:** Carry out user testing and incorporate feedback into the final system.
- **Weeks 10–11:** Complete final system evaluation, report writing, and submission preparation.

These milestones are designed to build on the existing system while allowing sufficient time for testing, evaluation, and refinement.

1.3 Reflection

Term One provided valuable insight into both the technical and organisational challenges of developing a full-stack system.

- **What went well:**
 - The early adoption of Test Driven Development significantly improved backend reliability and code quality.
 - A clear separation between frontend and backend resulted in a maintainable and scalable architecture.
 - Firebase authentication proved to be a secure and effective solution for user management.
 - End-to-end functionality was successfully achieved for core features.
 - Regular meetings with the supervisor provided structure, motivation, and accountability.
 - Strong use of software engineering practices, including CI pipelines, version control, and testing.
 - Backend package structure and layering were well organised and easy to navigate.
- **What did not go well:**
 - The complexity of security configuration and token handling was underestimated.

- Frontend design required more iteration than planned due to limited prior UI experience.
- Poor time management at the beginning of the term caused early delays.
- Large commits made it harder to track changes and identify regressions.
- Infrequent diary updates early on made progress harder to monitor.
- Initial frontend package organisation lacked clarity and required restructuring.

- **Improvements made:**

- Time management improved significantly in the latter half of the term.
- Frontend design and structuring skills developed through iterative refactoring.
- Better separation of public and private pages improved maintainability.
- Authentication and API integration issues were resolved through systematic debugging.
- Communication between frontend and backend became more efficient and consistent.

Overall, Term One highlighted the importance of realistic planning, incremental development, and maintaining consistent project documentation. These lessons will directly inform Term Two and reduce the likelihood of repeating earlier mistakes.

1.4 Risk Assessment

- **Security Risks:** As the system handles sensitive user and financial data, security was a primary concern. These risks were mitigated through Firebase Authentication, protected frontend routes, and backend token validation using a dedicated security filter. Configuration files were securely managed to prevent credential exposure, and authentication and security configuration were covered by targeted tests to reduce the risk of unauthorised access.
- **Performance Risks:** Performance risks were considered early due to the potential impact of frequent API requests and data-intensive dashboard views. Efficient entity design, RESTful API separation, and clear service-layer boundaries helped minimise bottlenecks. Performance profiling and optimisation are planned for Term Two as real-time data integration is introduced.

- **Browser Compatibility Risks:** Compatibility risks were reduced by using standard React tooling, modern web APIs, and responsive layout techniques. Regular manual testing was performed using contemporary browsers to ensure consistent behaviour and layout across devices.
- **Project Delay Risks:** The risk of project delays was mitigated through incremental development, small and frequent commits, and feature-based branching. Automated GitLab CI pipelines enforced build quality and testing, while regular supervisor meetings provided guidance and helped realign priorities when delays occurred.

Overall, the risks identified in the project plan closely reflected the challenges encountered during development. The mitigation strategies proved effective during Term One and provide a strong framework for managing risk during the final phase of the project.